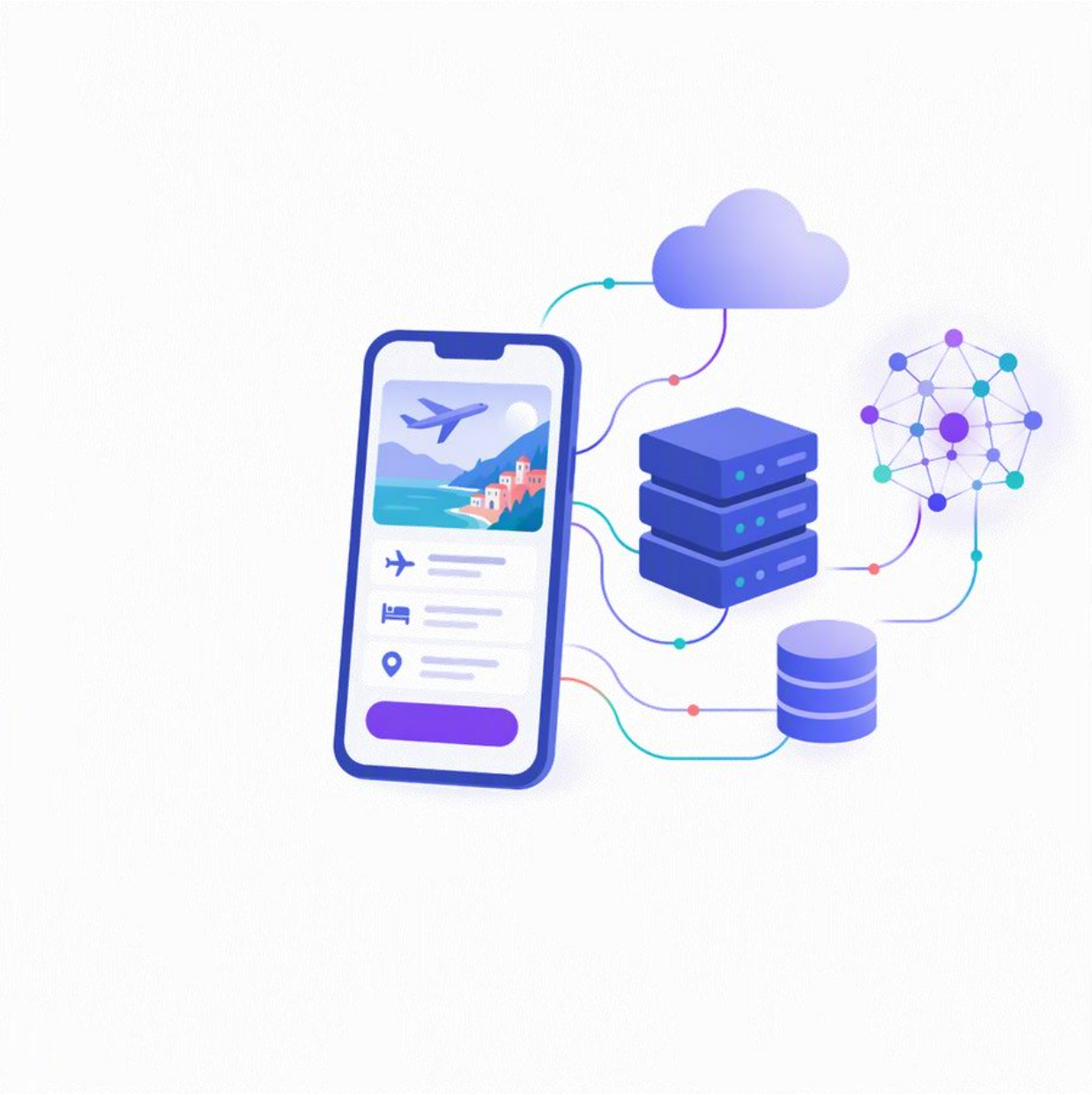
PRODUCT MANAGEMENT 101 · SESSION 6

# Tech for PMs

How can PMs make strategic decisions without being engineers?



OPENER · 8 MIN

# Reflection: tech fluency ≠ coding

PRE-CLASS REFLECTION

## “How can tech fluency make a PM more strategic?”

Pairs: 2 min to compare answers. Then three voices to the room.

**01** **Better trade-offs**
Know what is cheap vs expensive to build — and why an estimate is big.

**02** **A credible partner**
Engineers engage more deeply with PMs who understand the system.

**03** **Spot leverage**
APIs, data and AI open options that rivals miss.

AGENDA · 120 MIN

# Today: two cases, five modules, five exercises



0 min 120 min

| | | |
|---|---|---|
| 0–8 | **Opener** | Reflection · tech fluency ≠ coding |
| 8–20 | **01 How software works** | Tap “Book”, layers, data · Case 1 |
| 20–30 | **Hands-on 1** | Draw the request |
| 30–43 | **02 APIs & architecture** | APIs, webhooks, system · Case 1 |
| 43–55 | **Hands-on 2** | Write the API |
| 55–62 | **03 Workflows → algorithms** | Booking flow → algorithm · Case 1 |
| 62–74 | **Hands-on 3** | Workflow → algorithm |
| 74–82 | **04 How teams ship** | Waterfall, Agile, Scrum, stories |
| 82–94 | **05 Intro to AI & ML** | ML, training, precision/recall |
| 94–102 | **Hands-on 4** | Is this an ML problem? |
| 102–117 | **Hands-on 5** | LLMs, RAG, build vs buy · Case 2 |
| 117–120 | **Wrap-up** | Questions · takeaways · take-home |

# The Journey of a URL

A URL (Uniform Resource Locator) tells your browser where to find a resource on the internet.
It is made up of several parts, each with a specific purpose.

## Anatomy of a URL

`https://` `example.com` `/product/electric/phone` `?ref=ad&sort=price`

**Scheme**
Defines the protocol (HTTP, HTTPS, WS, WSS, etc.)

**Domain**
The website or server you want to reach.

**Path**
The specific location on the server (e.g., folder or endpoint).

**Query Parameters**
Additional key-value pairs (e.g., ref=ad, sort=price).

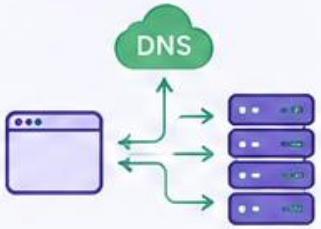
### 1 You enter the URL




Your browser parses the URL and identifies the scheme, domain, path and query parameters.

- The browser also checks its cache for a recent DNS entry.

### 2 DNS Resolution




The domain name is converted to an IP address through the DNS lookup process (root → TLD → authoritative server).

- The IP is cached at multiple layers (browser, OS, ISP) for faster lookups.

### 3 Establish TCP Connection




Using the IP address, your browser establishes a TCP connection with the server (or load balancer).

- This ensures a reliable connection before sending the actual request.

### 4 Send HTTP Request

```
GET /product/electric/phone
?ref=ad&sort=price HTTP/1.1
```

Headers:
- Host: example.com
- User-Agent: ...
- Accept: application/json

Your browser sends an HTTP request over the TCP connection using the correct format (method, headers, body).

- The server understands this request and knows what you want.

### 5 Server Processing




The request is routed through load balancers, app servers, cache and (if needed) the database to fetch the data.

- Each layer helps with scalability, speed and reliability.

### 6 Response & Render

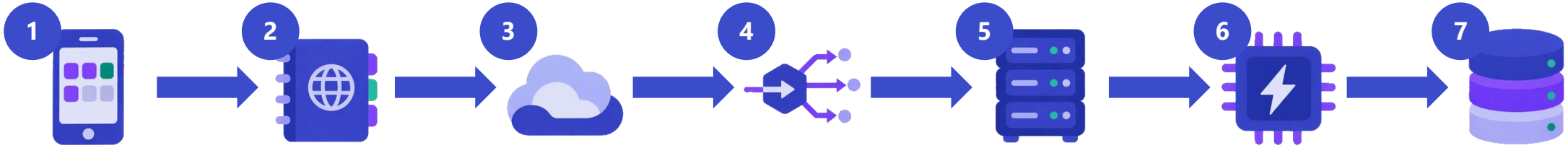



The server sends back an HTTP response (e.g., 200 OK) with the data (usually in JSON). Your browser receives it and renders the page.

- If something goes wrong (e.g., timeout or 404), you'll see an error instead.

CASE STUDY 1

# What happens when you tap “Book”




| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|
| **Client** | **DNS** | **Internet** | **Load balancer** | **App server** | **Cache** | **Database** |
| You tap Book | Name → IP | Routers relay | Picks a server | Runs the logic | Hot data, fast | Saves booking |

**Response: 200 OK + JSON → app shows “Booking confirmed”** 8

Request path  Response path

**PM INSIGHT**

Every product = interface + data + logic. Every hop adds delay — when users say “it’s slow”, ask which hop.

CASE STUDY 1

# Front-end, back-end, database

FRONT-END
**What the user sees**
Date picker, price, Book button



TECH STACK
React · JavaScript · Swift (iOS) · Kotlin

CALL
**Front-end team**
e.g. button misaligned

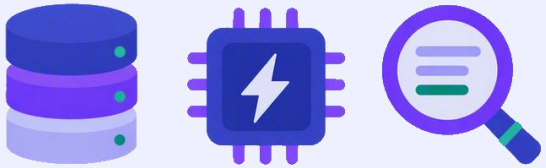
BACK-END
**What runs on servers**
Checks dates, totals price



TECH STACK
Java · Python · Node.js · Go · Rails

CALL
**Back-end team**
e.g. wrong total, slow

DATABASE
**Where data lives**
Listings, users, bookings



TECH STACK
PostgreSQL · MySQL · Redis · Elasticsearch

CALL
**Data / infra team**
e.g. slow search

CASE STUDY 1

# The data model behind a booking

### USERS

| user_id (PK) | name | city |
|---|---|---|
| U201 | Priya | Mumbai |
| U202 | Liam | London |

### LISTINGS

| listing_id (PK) | host_id (FK → Users) | city | price / night |
|---|---|---|---|
| L101 | U202 | Lisbon | €120 |
| L102 | U202 | London | £150 |

### BOOKINGS

| booking_id (PK) | listing_id (FK) | guest_id (FK) | dates | status |
|---|---|---|---|---|
| B501 | L101 | U201 | 12–15 Dec | confirmed |
| B502 | L102 | U201 | 20–22 Jan | pending |

**KEYS**

PK — primary key: the unique ID of each row.
FK — foreign key: points to a row in another table.

A “small” feature like split payments needs a new Payments table that’s why it isn’t a one-day job.

HANDS-ON 1 · 10 MIN

# 01

# Draw the request

Map one tap across the whole system.

**TASK**
Pick one tap — “Book” on Airbnb or “Confirm ride” on Uber. Draw its journey: client → DNS → API → back-end → database → third parties (payments, maps, SMS) → back to the screen.

**GROUPS**
2people · one sheet or whiteboard per group

**TIME**
7 min to draw · 3 min gallery walk

**DELIVERABLE**
One diagram with every hop labelled, plus the one hop you think fails most often

**DEBRIEF**
Where would a 2-second delay hurt most — and which team would you call?

CASE STUDY 1

# Search listings: a GET request

1 · REQUEST · GET

```
GET /v1/listings?city=Lisbon&check_in=2026-12-12
     &check_out=2026-12-15&guests=2
```



Airbnb app



Airbnb API

2 · RESPONSE · 200 OK

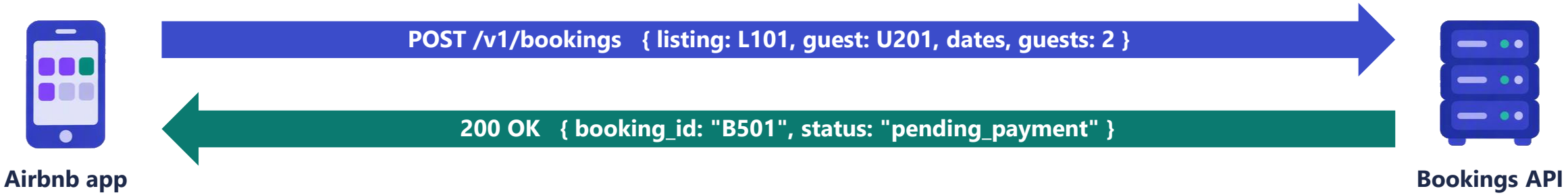
```
{ "results": [
    { "listing_id": "L101", "title": "Sunny loft, Alfama",
      "price_per_night": 120, "currency": "EUR", "rating": 4.9 },
    ... ],  "total": 214 }
```

**PM INSIGHT** Request = endpoint + method + auth + filters; response = status + JSON.

Illustrative API for teaching — not Airbnb's real endpoint.

CASE STUDY 1

# Create a booking: POST + status codes




## STATUS CODES · WHAT THE USER SEES

| 200 | 400 | 401 | 404 | 429 | 500 |
|---|---|---|---|---|---|
| **Booking created** | **Bad dates** | **Login expired** | **Listing removed** | **Too many taps** | **Our bug** |
| "Almost done — pay" | Inline date error | Ask to log in | Show similar stays | Retry quietly | Apologise + retry |

**PM INSIGHT** Error states are product decisions: spec the message for every code.

CASE STUDY 1

# Don’t build it all: third-party APIs



**Airbnb back-end**
your code

1 · Charge €360 for B501

2 · Webhook: payment succeeded



PAYMENTS
**Stripe**
Charge, refund, fraud checks

Show the listing on a map



MAPS
**Google Maps**
Map + distance to the beach

3 · Text the host “New booking!”

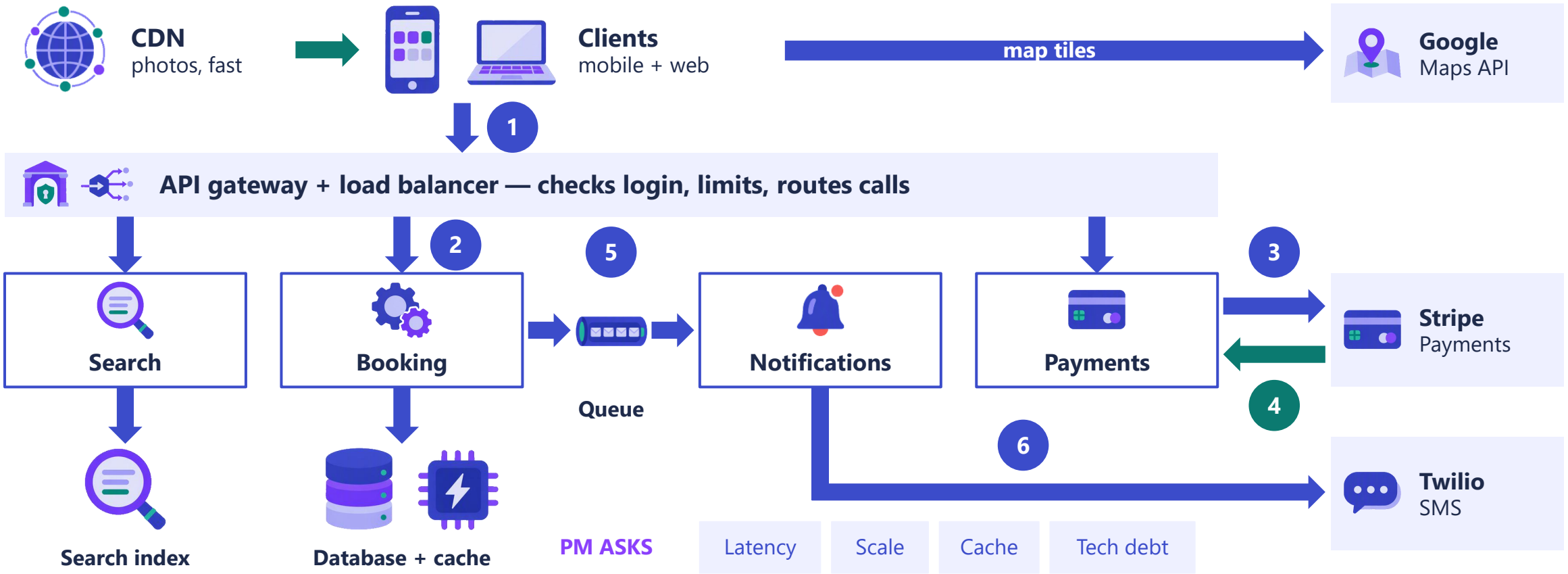


SMS
**Twilio**
Text the host, login codes

**PM INSIGHT** API = you call them. Webhook = they call you — no “paid yet?” polling.

CASE STUDY 1

# The whole system behind “Book”




1 Tap Book  2 Booking checks DB + cache  3 Charge card  4 Webhook back  5 Event queued  6 SMS to host

Request  Response / webhook

Illustrative — real Airbnb runs many more services.

HANDS-ON 2 · 12 MIN

# 02

# Write the API

Turn a feature into a contract.

**TASK**
Design the API for “Save to wishlist” (the heart icon on an Airbnb listing). Write the endpoint and method, the request payload in JSON, the success response, and two error responses.

**GROUPS**
3–4 people

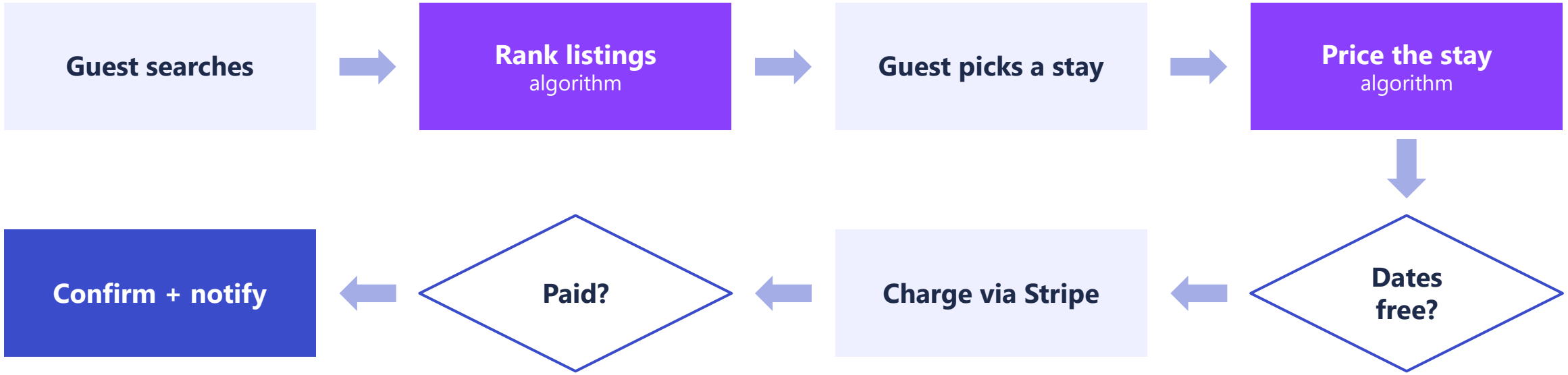
**TIME**
8 min to write · 4 min swap with a neighbour and critique

**DELIVERABLE**
One API card: e.g. POST /v1/wishlists/{id}/items with 200, 401 and 404 bodies

**DEBRIEF**
What did writing the API force you to decide that the feature brief never said?

CASE STUDY 1

# The booking workflow, step by step




No → show similar stays

No → retry or another card

= an algorithm decides here: inputs → logic → output → metric

**PM INSIGHT**

PMs own the workflow and the success metric; engineering and data science own the logic inside the violet boxes.

HANDS-ON 3 · 12 MIN

# 03

# Workflow → algorithm

Define the decision before the model.

**TASK**
Pick one: Uber driver matching or a Spotify “Discover Weekly” playlist. Define the inputs (data), the logic (the rules you would write, or “a model learns from…”), the output, ONE success metric and ONE guardrail metric.

**GROUPS**
4 people

**TIME**
8 min to build · 4 min for two groups to share

**DELIVERABLE**
A four-box card: Inputs → Logic → Output → Metric

**DEBRIEF**
What could the algorithm optimise that would hurt users or partners?

# Waterfall vs Agile vs Scrum

| | Waterfall | Agile | Scrum |
|---|---|---|---|
| How work flows | Requirements → design → build → test → release, in sequence | Small increments with continuous feedback | An Agile framework run in fixed sprints of 1–4 weeks |
| When change is cheap | Only at the start | Any time | Between sprints |
| Best for | Fixed scope, regulated or hardware work | Uncertain needs, consumer software | Teams that need a rhythm and clear roles |
| PM's role | Write the full spec up front | Prioritise continuously | Product Owner: owns the backlog |
| Seen at | Large regulated programmes, e.g. core-banking replacement | Spotify's autonomous squads | Teams running sprints on Atlassian's Jira |

**PM INSIGHT**

Most companies run a hybrid: Agile delivery inside a roadmap that still has fixed launch dates. Learn the principles, not the ritual.

# Scrum in one picture




## WHO'S WHO



**Product Owner**
the PM: owns what & why



**Scrum Master**
Runs rituals, clears blockers



**Dev team**
Owns the how; estimates



**Story points**
1, 2, 3, 5, 8, 13 — size, not hours

CASE STUDY 1

# Write a user story engineers love

USER STORY

**As a guest planning a trip with friends, I want to save listings to a wishlist, so that we can compare them later.**

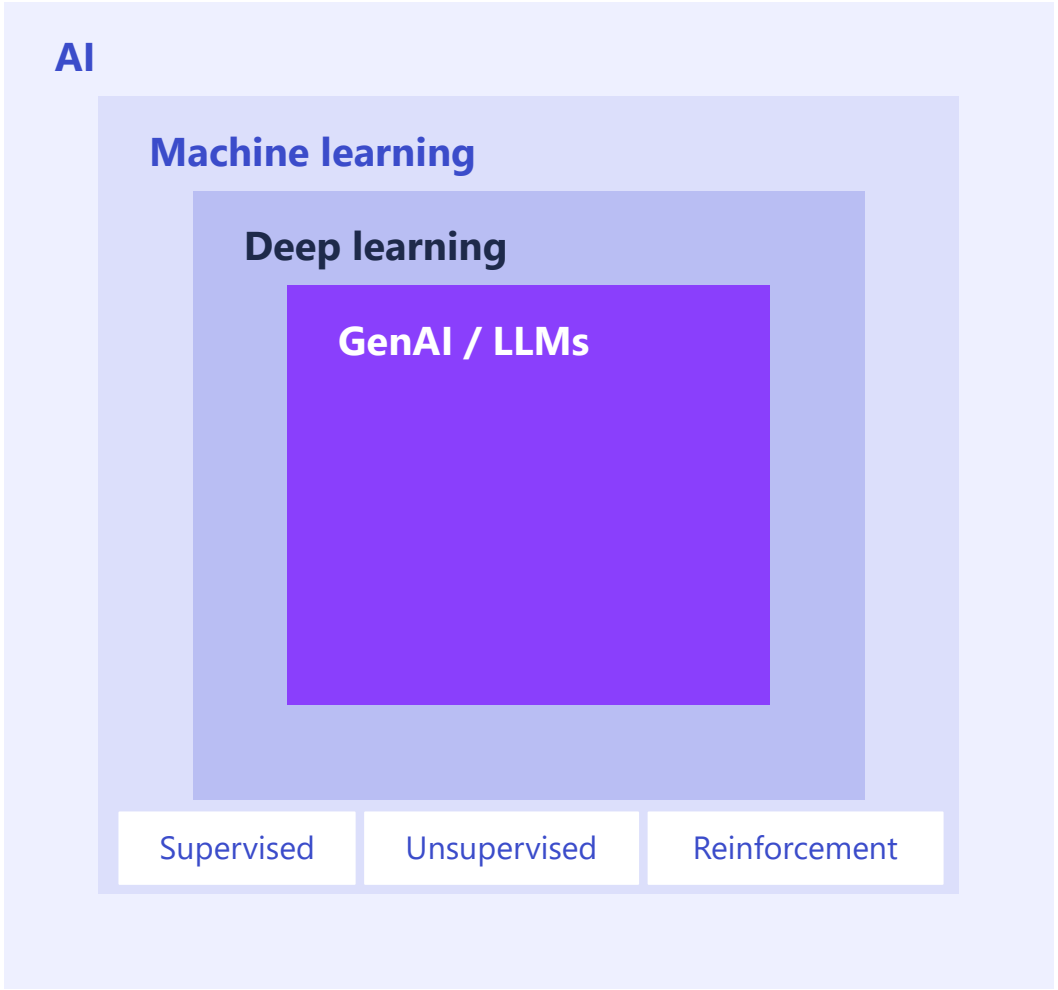
**Estimate: 5 story points**

ACCEPTANCE CRITERIA · GIVEN / WHEN / THEN

1. Given I am logged in, when I tap ♡ on a listing, then it is saved to my wishlist within 1 second.
2. Given I am logged out, when I tap ♡, then I am asked to log in and the save completes afterwards.
3. Given a saved listing is removed, when I open my wishlist, then it shows “No longer available”.
4. Given I tap ♡ twice quickly, then only one item is saved.

INVEST check — a good story is Independent · Negotiable · Valuable · Estimable · Small · Testable

# AI ⊃ ML ⊃ deep learning ⊃ GenAI




## AI

Machines doing tasks that seem intelligent — including hand-written rules.

**e.g. Chess engines, route planning**

## Machine learning

Learns patterns from data instead of rules.

**e.g. Netflix recommendations, Stripe Radar**

## Deep learning

ML with many-layered neural networks; strong on images, speech, text.

**e.g. Face ID, Google Translate**

## GenAI / LLMs

Deep learning that generates new text, images or code.

**e.g. ChatGPT, GitHub Copilot, Notion AI**

# How ML works: train once, infer often

## TRAINING · OFFLINE, RE-RUN PERIODICALLY



**Past payments**
millions, with features

**Labels**
fraud? yes / no

**Model learns**
the fraud patterns

deploy

## INFERENCE · LIVE, ON EVERY PAYMENT



**New payment**
arrives at checkout

**Same features**
computed in ms

**Model**
returns a risk score

**Fraud score**
allow · verify · block

**PM INSIGHT**
Features are the clues; labels are the answers.

Example: $980 · US card · IP in Romania · new device → score 0.92 → ask to verify (illustrative)

# When the model is wrong: precision & recall

FRAUD MODEL ON 10,000 PAYMENTS · ILLUSTRATIVE

| | Actually fraud | Actually legit |
|---|---|---|
| Model flags fraud | True positive: 80 — fraud caught | False positive: 20 — good customer blocked |
| Model says legit | False negative: 40 — fraud slipped through | True negative: 9,860 — correct pass |

**PM INSIGHT**

Raise the threshold → fewer false positives (happier customers) but more missed fraud. For Gmail, a real email lost in spam hurts more than one spam that gets through. Choosing the trade-off is a product decision.

**80%**

**Precision**

Of what we flagged, how much was really fraud? 80 ÷ (80 + 20)

**67%**

**Recall**

Of all real fraud, how much did we catch? 80 ÷ (80 + 40)

HANDS-ON 4 · 8 MIN

# 04

# Is this an ML problem?

**TASK**

Whole class, rapid-fire. Vote on each with fingers: 1 = Rules · 2 = Classic ML · 3 = GenAI. 45 seconds each.

**DEBRIEF**

Which ones split the room — and why?

1. Block sign-ups from disposable email domains
2. Predict which hosts will stop listing next month
3. Draft a reply to an angry customer email
4. Add the correct sales tax at checkout
5. Recommend the next show to watch
6. Flag possibly fraudulent payments
7. Summarise a 40-minute sales call
8. Send a reminder 24 hours before check-in

CASE STUDY 2

# LLM basics every PM needs

### Tokens
Chunks of text, roughly ¾ of an English word. You pay per token in and out.

### Prompt
The instructions + context you send. Much product behaviour lives here.

### Context window
How much text the model can consider at once. Long histories may not fit.

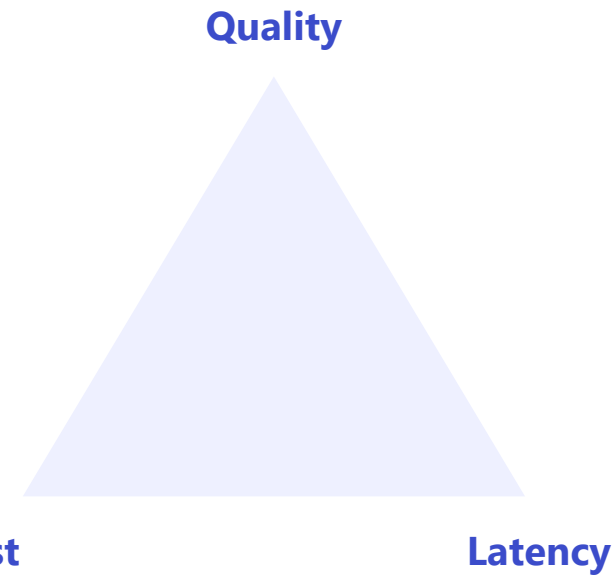
### Hallucination
A confident but wrong answer. Design for it; never assume it away.

### RAG
Retrieve relevant documents first, then answer from them — like Intercom Fin.

### Fine-tuning
Further training on your examples to shape tone or format. Slower to change.

**Case 2: HelpDeskly (fictional) wants an AI support assistant**

THE TRADE-OFF TRIANGLE




Bigger models answer better but cost more and respond slower. Route easy questions to small models.

CASE STUDY 2

# How the AI assistant answers: RAG




**PM INSIGHT**

Ground answers in your docs, test on a golden set, add guardrails — you own what your bot says (Air Canada, 2024).

CASE STUDY 2

# Build, buy, or call an API?

| Criterion | Model API (OpenAI, Anthropic, Azure OpenAI, Gemini) | Open-source model, self-hosted (Llama, Mistral) | Train in-house | Buy a SaaS agent (Intercom Fin, Zendesk AI) |
|---|---|---|---|---|
| Speed to ship | Days to weeks | Weeks to months | Months or more | Days |
| Cost | Pay per token | Infrastructure + ML engineers | Highest: data, talent, GPUs | Subscription or usage fee |
| Control | Medium | High | Highest | Low |
| Data privacy | Depends on vendor terms | Data stays in your cloud | Full | Vendor holds your data |
| Differentiation | Prompts, data and UX | Medium | High if your data is unique | Low — rivals can buy it too |
| Quality risk | Strong models; vendor may change them | You own tuning and evals | Hard to match frontier models | Vendor roadmap decides |

**TAKE-HOME LINK**

Startup default: buy or call an API to learn fast; build only where your data or workflow is the moat.

HANDS-ON 5 · 15 MIN

# 05

# Build vs buy

Decide for the AI assistant — and defend it in 60 seconds.

**TASK**
HelpDeskly has 6 engineers, no ML team and enterprise clients in banking. Score three options — Model API · Open-source self-hosted · SaaS agent — from 1 to 5 on speed, cost, control, data privacy, differentiation and quality risk. Weight what matters most, then decide.

**GROUPS**
5–6 people

**TIME**
10 min to score · 5 min for three 60-second pitches

**DELIVERABLE**
Filled matrix + a 60-second pitch: choice, why, biggest risk, how you would reduce it

**DEBRIEF**
Which criterion flipped your decision? Would it change with 600 engineers?

CHEAT SHEET

# Questions every PM should ask engineering

| Layer | Ask engineering | Usually owned by |
|---|---|---|
| Front-end | How does this behave on slow networks, small screens and offline? | Web / iOS / Android team |
| API | What are the error states — and what does the user see for each? | Back-end / platform |
| Back-end & data | What new tables or fields does this need? Any data migration? | Back-end, data engineering |
| Third parties | What if Stripe or Twilio is down? Fallback? Cost per call? | Payments / integrations |
| Speed & scale | What breaks at 10× traffic? What is our latency budget? | Infrastructure / SRE |
| Tech debt | What shortcut are we taking, and when do we repay it? | Tech lead |
| AI / ML | How will we measure quality, and what happens when it is wrong? | ML / applied AI team |

**WHEN TO ASK**

Ask early — in backlog refinement, not at sprint review. One good question can save a sprint.

WRAP-UP

# Key takeaways

**01** Tech fluency is about trade-offs, not code.

**02** Every tap is a chain: client → API → back-end → data → third parties.

**03** APIs are contracts — spec the errors, not just the happy path.

**04** You own the workflow; define each algorithm's inputs, output and metric.

**05** Scrum is a rhythm for learning; a good story makes "done" testable.

**06** For AI: start from the problem, measure quality, add guardrails — then decide build vs buy.



BEFORE NEXT CLASS

# Take-home, watch, reflect

OUTCLASS · STARTUP QUESTION

## “If your tech team is small, how would you balance building in-house vs using third-party tools or APIs to ship faster?”

One page. Use the build-vs-buy matrix and name one real company as your example.

POST-CLASS WATCH

### Andrew Ng
### “Opportunities in AI”

youtube.com/watch?v=5p248yoa3oE
Note one AI opportunity he names that fits a product you use.

NEXT · SESSION 7

### Analytics for Product Strategy

**Pre-class reflection:**
“Which metric best represents user success?”